LAPORAN PRAKTIKUM

POSTTEST 2

ALGORITMA PEMROGRAMAN LANJUTAN

Disusun oleh:

Luvita Khairana Salwa (2509106041)

Kelas A'25

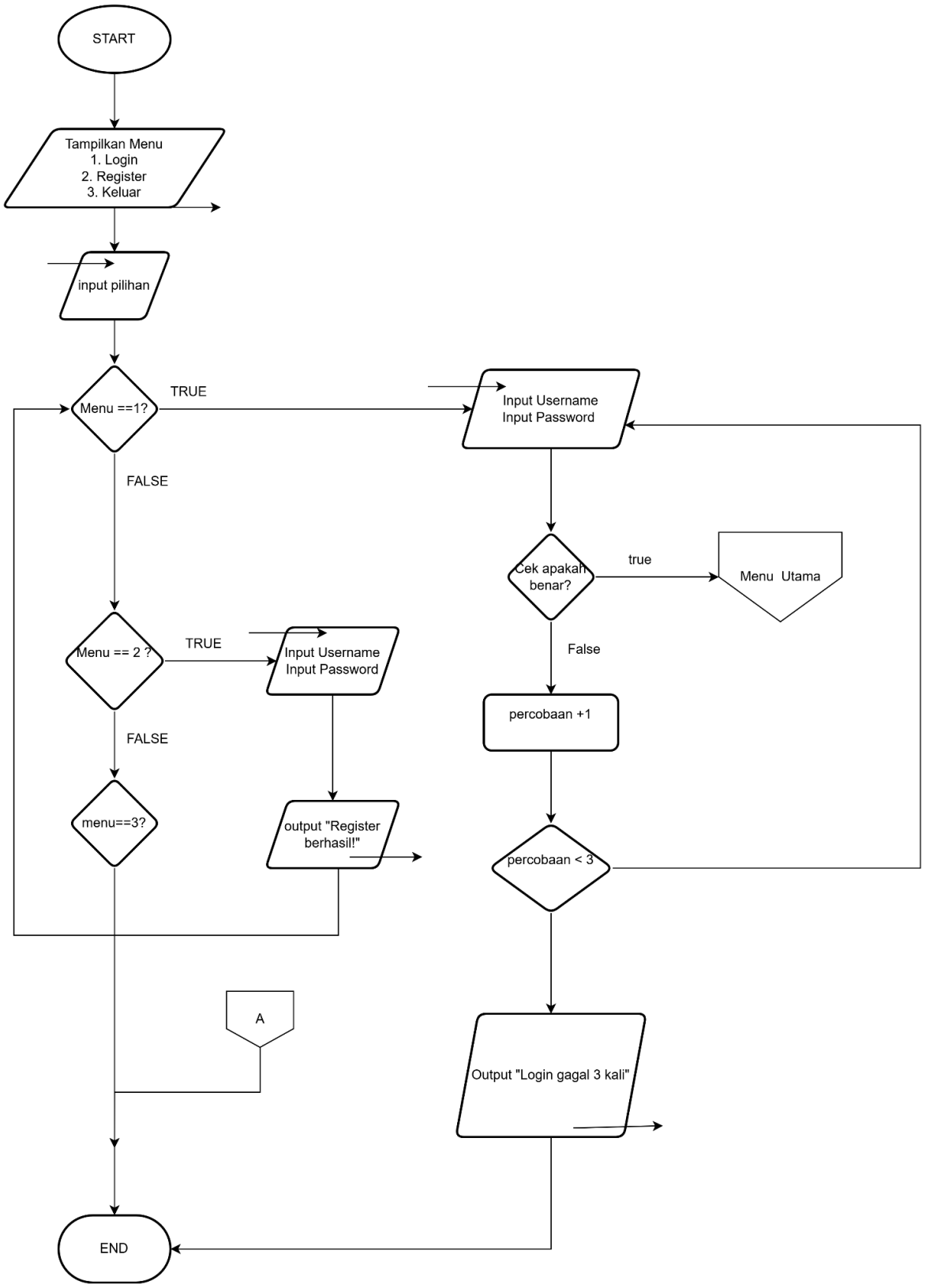
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart



1.1 Menu Login

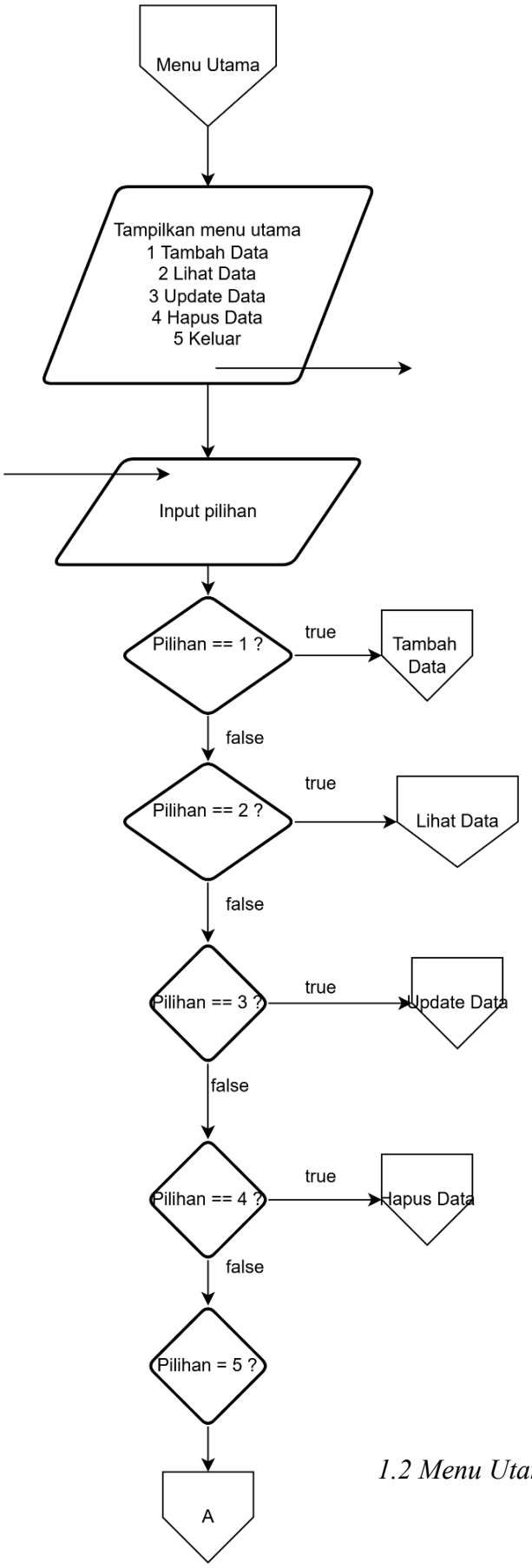
Penjelasan gambar 1.1 Menu login

Program dimulai dengan menampilkan menu yang berisi tiga pilihan yaitu Login, Register, dan Keluar. Pengguna kemudian diminta memasukkan pilihan menu sesuai dengan nomor yang tersedia. Jika pengguna memilih **Login,** sistem akan meminta pengguna untuk memasukkan username dan password. Setelah itu sistem akan memeriksa apakah username dan password yang dimasukkan sesuai dengan data yang tersimpan di dalam sistem. Jika data yang dimasukkan benar, maka pengguna berhasil login dan akan masuk ke menu utama.

Namun jika username atau password yang dimasukkan salah, pengguna masih diberikan kesempatan untuk mencoba login kembali. Proses login ini dibatasi maksimal tiga kali percobaan. Jika pengguna berhasil login sebelum tiga kali percobaan, maka pengguna dapat masuk ke menu utama. Tetapi jika pengguna gagal login sebanyak tiga kali, maka sistem akan langsung menghentikan program dan proses berakhir.

Jika pengguna memilih **Register,** sistem akan meminta pengguna untuk memasukkan username dan password baru. Data yang dimasukkan kemudian disimpan ke dalam sistem sebagai data akun baru. Setelah proses pendaftaran berhasil, pengguna akan langsung diarahkan kembali ke proses login untuk masuk menggunakan akun yang baru saja dibuat. Pada tahap login ini, pengguna juga hanya diberikan kesempatan maksimal tiga kali percobaan. Jika dalam tiga kali percobaan login masih gagal, maka program akan langsung berakhir.

Jika pengguna memilih **Keluar**, maka sistem akan langsung menghentikan program dan proses selesai tanpa menjalankan proses lainnya.



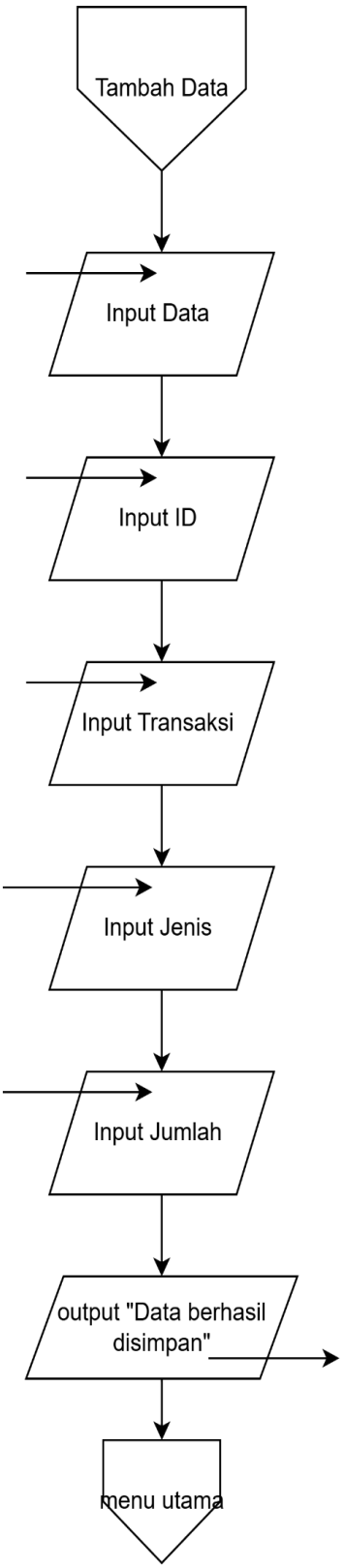
1.2 Menu Utama (CRUD)

Penjelasan 1.2 Menu Utama (CRUD)

Setelah pengguna berhasil login, sistem menampilkan **Menu Utama** yang berisi beberapa pilihan yaitu Tambah Data, Lihat Data, Update Data, Hapus Data, dan Keluar. Pengguna kemudian memilih salah satu menu.

Jika memilih **Tambah Data**, pengguna diminta memasukkan ID, nama transaksi, jenis transaksi (masuk atau keluar), dan jumlah uang. Data tersebut kemudian disimpan ke dalam sistem. Jika memilih **Lihat Data**, sistem akan menampilkan seluruh data keuangan yang telah dimasukkan, serta menghitung total pemasukan, total pengeluaran, dan saldo akhir.

Jika memilih **Update Data**, pengguna diminta memasukkan ID data yang ingin diubah. Jika ID ditemukan, pengguna dapat mengganti transaksi, jenis, dan jumlah data. Jika tidak ditemukan, sistem menampilkan pesan bahwa data tidak ditemukan. Jika memilih **Hapus Data**, pengguna memasukkan ID data yang ingin dihapus. Jika data ditemukan, sistem akan menghapus data tersebut. Jika tidak ditemukan, sistem menampilkan pesan bahwa data tidak ditemukan. Jika memilih **Keluar**, program berhenti dan proses selesai.

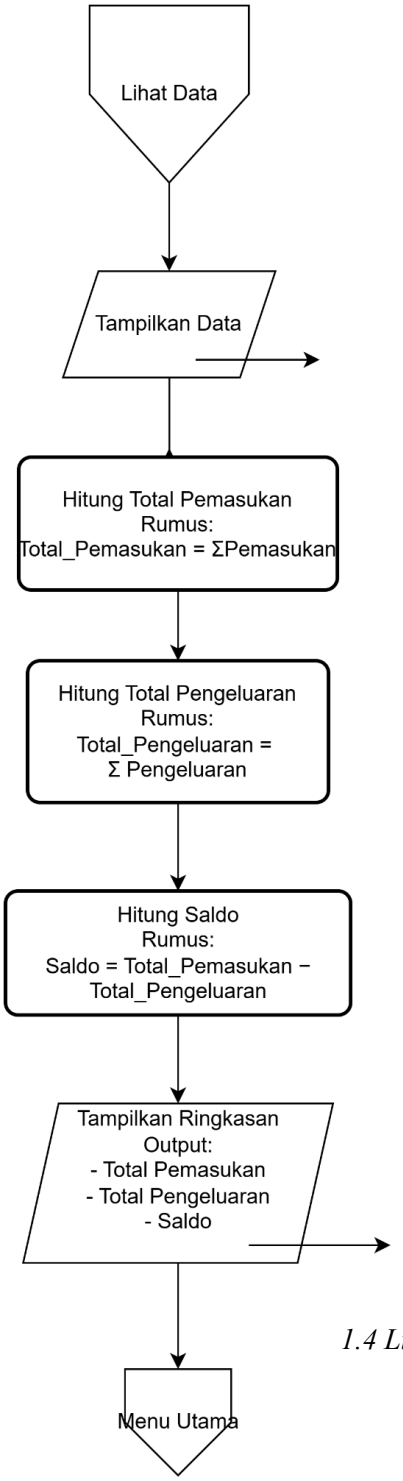


1.3 Flowchart tambah data (create)

Penjelasan 1.3 Flowchart tambah data (Create)

Pada menu utama, jika pengguna memilih **Tambah Data**, sistem akan meminta pengguna memasukkan ID transaksi. Setelah itu pengguna memasukkan nama transaksi, jenis transaksi (masuk atau keluar), dan jumlah uang.

Setelah semua data dimasukkan, sistem akan menyimpan data ke dalam array data keuangan. Kemudian jumlah data bertambah satu. Setelah data berhasil disimpan, sistem akan kembali ke Menu Utama untuk menunggu pilihan berikutnya dari pengguna.

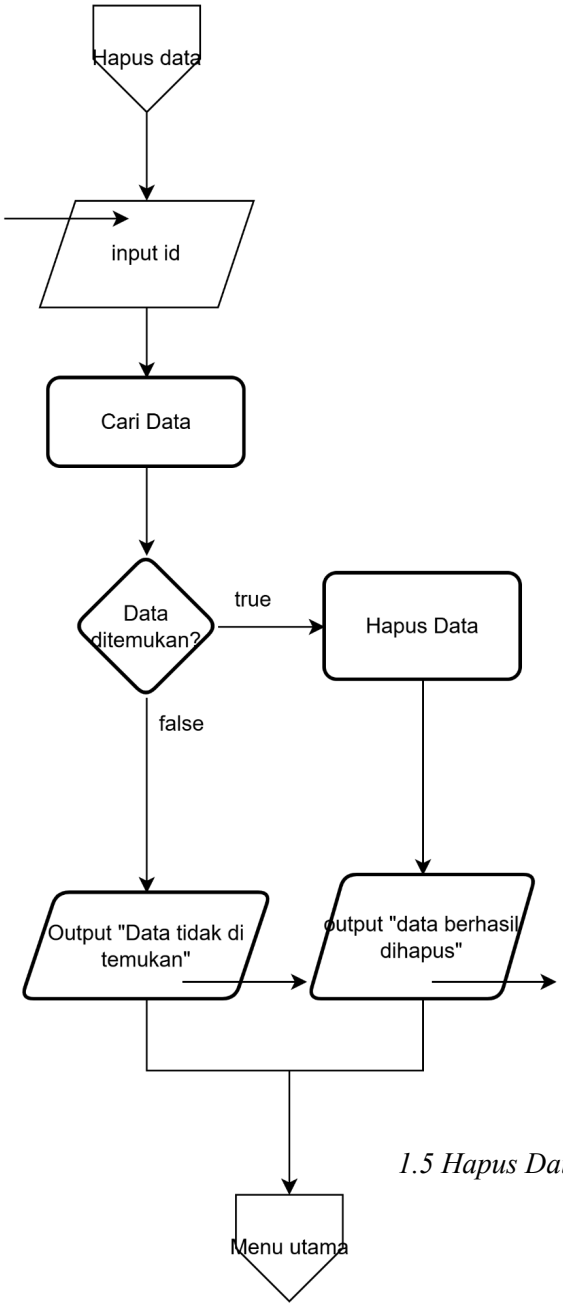


1.4 Lihat Data (Read)

Penjelasan 1.4 Lihat Data (Read)

Pada menu utama, jika pengguna memilih **Lihat Data**, sistem akan memeriksa apakah ada data yang tersimpan. Jika belum ada data, sistem akan menampilkan pesan “Belum ada data”. Jika ada data, sistem akan menampilkan seluruh data keuangan yang meliputi ID, nama transaksi, jenis transaksi, dan jumlah. Selama menampilkan data, sistem juga menghitung total pemasukan dan total pengeluaran.

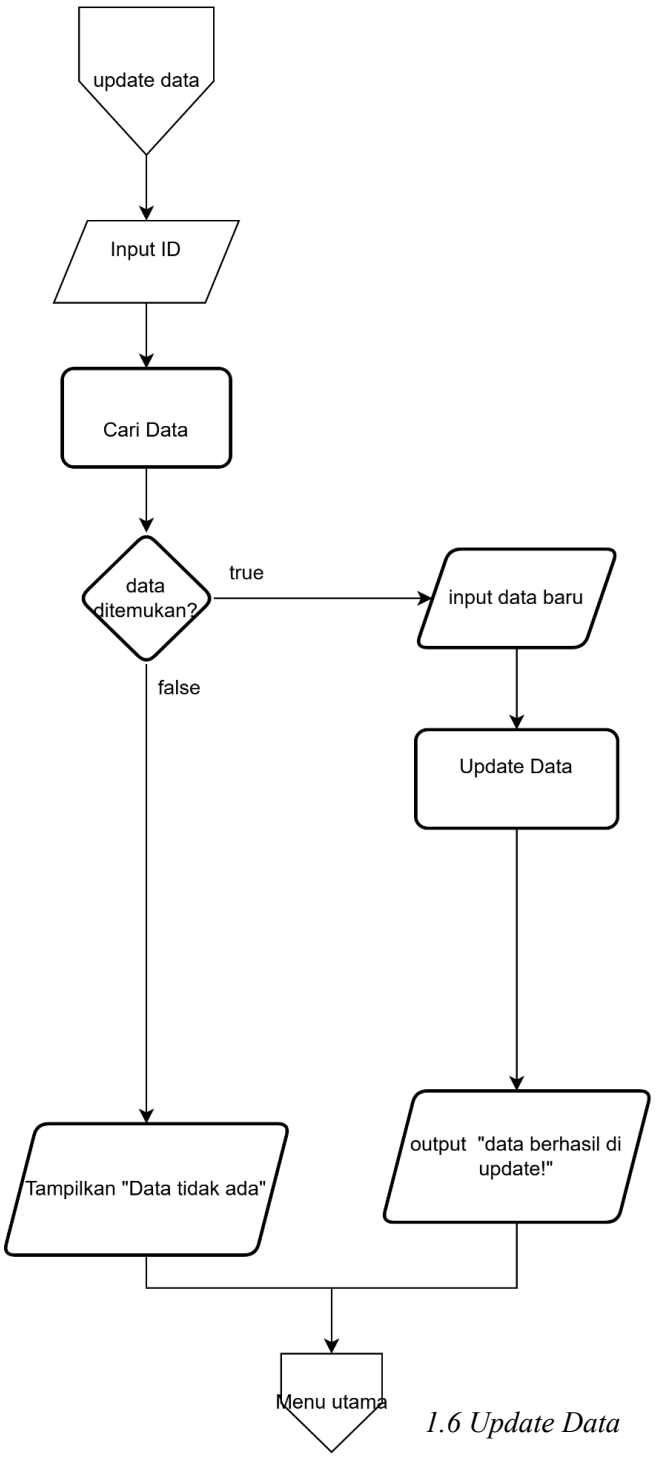
Setelah semua data ditampilkan, sistem menghitung saldo akhir dengan rumus total pemasukan dikurangi total pengeluaran. Kemudian hasil total pemasukan, total pengeluaran, dan saldo akhir ditampilkan, lalu program kembali ke Menu Utama.



1.5 Hapus Data (Delete)

Penjelasan 1.5 Hapus Data (Delete)

Pada menu utama, jika pengguna memilih **Hapus Data,** sistem akan meminta pengguna memasukkan ID data yang ingin dihapus. Sistem kemudian mencari ID tersebut pada data keuangan. Jika ID ditemukan, sistem akan menghapus data tersebut dari array dengan menggeser data setelahnya, lalu jumlah data dikurangi satu dan menampilkan pesan data berhasil dihapus. Jika ID tidak ditemukan, sistem akan menampilkan pesan data tidak ditemukan. Setelah proses selesai, program kembali ke Menu Utama.



1.6 Update Data

Penjelasan 1.6 Update Data

Pada menu utama, jika pengguna memilih **Update Data**, sistem akan meminta pengguna memasukkan ID data yang ingin diubah.Sistem kemudian mencari ID tersebut pada data keuangan. Jika ID ditemukan, sistem meminta pengguna memasukkan nama transaksi baru, jenis transaksi baru, dan jumlah baru. Setelah itu data akan diperbarui dan sistem menampilkan pesan data berhasil diupdate.

Jika ID tidak ditemukan, sistem akan menampilkan pesan data tidak ditemukan. Setelah proses selesai, program kembali ke Menu Utama**.**

2. Deskripsi Singkat Program

Program ini merupakan Sistem Manajemen Keuangan Perusahaan berbasis bahasa pemrograman C++ yang dibuat untuk membantu pengguna dalam mengelola data transaksi keuangan. Program ini menggunakan konsep CRUD (Create, Read, Update, Delete) untuk menambah, menampilkan, mengubah, dan menghapus data transaksi.

Program diawali dengan tampilan menu awal yang terdiri dari tiga pilihan yaitu Login, Register, dan Keluar. Pada menu Login, pengguna dapat masuk ke dalam sistem menggunakan akun utama yang telah tersedia yaitu username vita dan password 041. Selain itu, pengguna juga dapat login menggunakan akun yang telah dibuat melalui menu register.

Pada menu Register, pengguna dapat membuat akun baru dengan memasukkan username dan password yang diinginkan. Akun tersebut kemudian dapat digunakan untuk login ke dalam sistem.

Pada proses login, pengguna diberikan maksimal tiga kali percobaan untuk memasukkan username dan password yang benar. Jika pengguna gagal melakukan login sebanyak tiga kali, maka program akan otomatis berhenti.

Setelah berhasil login, pengguna akan masuk ke menu utama sistem yang menampilkan beberapa pilihan menu yaitu menambah data transaksi, melihat data transaksi, mengubah data transaksi, menghapus data transaksi, dan keluar dari program. Program akan terus menampilkan menu ini secara berulang hingga pengguna memilih opsi keluar.

Data transaksi yang disimpan meliputi ID transaksi, tanggal transaksi, jenis transaksi (pemasukan atau pengeluaran), kategori transaksi, keterangan transaksi, dan nominal uang. Program ini menggunakan struct dan nested struct untuk menyimpan data secara terstruktur serta menggunakan array of struct untuk menyimpan banyak data transaksi.

3. Source Code

```
//nested crud
struct Keuangan{
    int id;
    string transaksi;
    string jenis;
    int jumlah;
};

struct User{
    string nama;
    string password;

    Keuangan data[100];
    int jumlahData;
};
```

Ada struct User dan Struct Keuangan yang berguna menyimpan data.

1. User → menyimpan username & password
2. Keuangan → menyimpan data transaksi

```
User users[10]; // max user
    int jumlahUser = 1;

    // akun admin awal
    users[0].nama = "Vita";
    users[0].password = "041";

    int menuAwal;
    string username, password;
    bool loginBerhasil = false;
```

```
int percobaan = 0;
while(percobaan < 3)
```

1. Login hanya **3 kali kesempatan**
2. Kalau gagal:

```
if(!loginBerhasil){
    cout << "Kesempatan login habis.\n";
    return 0;
```

```
User users[10]; // max user
```

```
    int jumlahUser = 1;
    // akun admin awal
    users[0].nama = "Vita";
    users[0].password = "041";
    int menuAwal;
    string username, password;
    bool loginBerhasil = false;
```

1. User users[10]; Membuat array bertipe User untuk menyimpan maksimal 10 akun pengguna.
2. int jumlahUser = 1; Variabel untuk menghitung jumlah user yang ada, dimulai dari 1 karena sudah ada akun awal.
3. users[0].nama = "Vita";
   users[0].password = "041"; Menetapkan akun admin awal dengan username Vita dan password 041.
4. int menuAwal; Variabel untuk menyimpan pilihan menu awal (Login, Register, atau Keluar).
5. string username, password; Variabel untuk menyimpan input username dan password saat login.
6. bool loginBerhasil = false; Variabel boolean untuk mengecek apakah login berhasil atau tidak. Awalnya bernilai false (belum berhasil).

```
int totalMasuk = 0;
int totalKeluar = 0;
```

Membuat dua variabel untuk menyimpan total uang:

1) totalMasuk → total pemasukan
2) totalKeluar → total pengeluaran

Nilai awalnya 0.

```
for(int i = 0; i < jumlahData; i++){
```

Perulangan for digunakan untuk membaca semua data transaksi dari array data.

1) dimulai dari i = 0
2) sampai jumlahData

```
cout << " ID        : " << data[i].id << endl;
cout << " Transaksi : " << data[i].transaksi << endl;
cout << " Jenis     : " << data[i].jenis << endl;
cout << " Jumlah    : " << data[i].jumlah << endl;
```

Bagian ini menampilkan informasi transaksi:

1) ID transaksi
2) Nama transaksi
3) Jenis transaksi (masuk / keluar)

4) Jumlah uang

```
if(data[i].jenis == "masuk"){
    totalMasuk += data[i].jumlah;
}
else if(data[i].jenis == "keluar"){
    totalKeluar += data[i].jumlah;
}
```

Program mengecek jenis transaksi:

1. Jika masuk, jumlah uang ditambahkan ke totalMasuk
2. Jika keluar, jumlah uang ditambahkan ke totalKeluar

artinya ditambah dengan nilai sebelumnya.

```
totalMasuk = totalMasuk + data[i].jumlah
```

```
int saldo = totalMasuk - totalKeluar;
```

```
cout << " Total Pemasukan   : " << totalMasuk << endl;
cout << " Total Pengeluaran : " << totalKeluar << endl;
cout << " Saldo Akhir       : " << saldo << endl;
```

```
cin.ignore();
cin.get();

 setw(20)
```

1. cin.ignore() membersihkan sisa input
2. cin.get() menunggu user menekan Enter

Biasanya dipakai di akhir menu supaya program tidak langsung kembali ke menu utama

Setw berarti set width (mengatur lebar kolom output). Jadi setw(20) artinya memberi ruang 20 karakter untuk data yang dicetak setelahnya.

```
#include <iostream>
#include <iomanip>
#include <limits>
#include <cstdlib>
```

```
using namespace std;
```

1. #include <iostream>
   Digunakan untuk operasi input dan output standar, seperti cin (input dari keyboard) dan cout (menampilkan output ke layar).
2. #include <iomanip>
   Digunakan untuk mengatur format tampilan output, misalnya mengatur jumlah angka di belakang koma atau merapikan tampilan data.
3. #include <limits>
   Digunakan untuk mengetahui atau menggunakan batas nilai maksimum dan minimum dari suatu tipe data.
4. #include <cstdlib>
   Berisi fungsi-fungsi umum dari pustaka standar C, seperti fungsi untuk angka acak (rand()), menghentikan program (exit()), atau menjalankan perintah sistem.
5. using namespace std;
   Digunakan agar kita tidak perlu menuliskan std:: di depan fungsi atau objek dari library standar, seperti std::cout atau std::cin.

4. Hasil output

```
================================================
          SISTEM MANAJEMEN KEUANGAN
================================================
 1. Login
 2. Register
 3. Keluar
================================================
 Pilih menu : 2

================ REGISTER =======================
 Username : miaw
 Password : 055
Register berhasil! Silakan login.

================ LOGIN ==========================
 Username : miaw
 Password : 055
```

4.1 Menu login&register

```
=====================================================
                     MENU UTAMA
=====================================================
 1. Tambah Data
 2. Lihat Data
 3. Update Data
 4. Hapus Data
 5. Keluar
=====================================================
 Pilih menu :
```

4.2 Menu Utama

```
================ DATA KEUANGAN ==================
ID        Transaksi          Jenis          Jumlah
-------------------------------------------------------
1         atk                keluar         30000
201       gaji               masuk          200000
-------------------------------------------------------
Total Pemasukan   : 200000
Total Pengeluaran : 30000
Saldo Akhir       : 170000

Tekan enter untuk lanjut...
```

4.3 Lihat Data (READ)

```
================ TAMBAH DATA =====================
 ID              : 005
 Nama Transaksi: INVESTASI
 Jenis (masuk/keluar): masuk
 Jumlah          : 300000000
Data berhasil ditambahkan!

Tekan enter untuk lanjut...
```

4.4 Tambah Data (CREATE)

```
==================================================
 Pilih menu : 3
Masukkan ID yang ingin diubah : 001
Transaksi baru : makanan
Jenis baru     : masuk
Jumlah baru    : 200000
Data berhasil diupdate!

Tekan enter untuk lanjut...
```

4.5 Ubah Data (UPDATE)

```
 Pilih menu : 4
Masukkan ID yang ingin dihapus : 001
Data berhasil dihapus!

Tekan enter untuk lanjut...
```

4.6 Hapus Data (DELETE)

```
================ DATA KEUANGAN ==================
ID        Transaksi          Jenis          Jumlah
---------------------------------------------------------
201       gaji               masuk          200000
5         INVESTASI          masuk          300000000
---------------------------------------------------------
Total Pemasukan   : 300200000
Total Pengeluaran : 0
Saldo Akhir       : 300200000

Tekan enter untuk lanjut...
```

4.7 Hapus Data (DELETE)

```
5. Keluar
==========================================
Pilih menu : 5

Tekan enter untuk lanjut...

Program selesai.
```

4.8 Keluar

5. Langkah-langkah GIT

```
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git add ./pertemuan-2
fatal: pathspec './pertemuan-2' did not match any files
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git add ./kelas
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git commit -m "pertemuan2"
[main dc8fdda] pertemuan2
 1 file changed, 94 insertions(+)
 create mode 100644 kelas/pertemuan-2/pertemuan-2.cpp
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.17 KiB | 300.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miawicey/APL.git
   b059d12..dc8fdda  main -> main
```

5.0.1 Langkah langkah git

5.1 GIT Add

Git add itu fungsi nya menambah file. Sebenarnya git add “nama file” tapi bisa “git add .” jika ingin menambah file keseluruhan

```
git add ./pertemuan-2
iles
```

5.1.1 git add

5.2 GIT Commit

Berfungsi menyimpan file yang udah di add ke repo tadi kayak biar hasil yang udah di add, menjadi file permanen di repo git

git commit -m “perintah”

```
L> git add ./kelas
L> git commit -m "pertemuan2"
```

5.2.1 Git commit

5.3 GIT Push

Mengirim commit dari repo lokal ke repo github jadi yaaaa reponya akan muncul secara online dan dapat di lihat di repostory github

“Git push -u origin main”

```
PS C:\Users\miaw miaw\OneDrive\Dokumen\GIT-DIR-APL> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.17 KiB | 300.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miawisoy/APL.git
```

5.3.1 Git push